

Politechnika Łódzka



**IAPŁ**

INSTYTUT AUTOMATYKI  
POLITECHNIKI ŁÓDZKIEJ

Zakład sterowania robotów

# **ROBOTY MOBILNE**

## **Instrukcja laboratoryjna 4**

### **Uruchomienie i obsługa środowiska**



# Spis treści

|          |                                                                                           |           |
|----------|-------------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Wprowadzenie</b>                                                                       | <b>2</b>  |
| 1.1      | Przydatne skróty klawiaturowe . . . . .                                                   | 2         |
| 1.2      | Współdzielenie zasobów pomiędzy komputerem natywnym a środowiskiem w kontenerze . . . . . | 2         |
| 1.2.1    | Wspólny katalog . . . . .                                                                 | 2         |
| 1.2.2    | Organizacja pracy studentów . . . . .                                                     | 3         |
| <b>2</b> | <b>Uruchomienie kontenera</b>                                                             | <b>4</b>  |
| 2.1      | Logowanie do systemu . . . . .                                                            | 4         |
| 2.2      | Połączenia sieciowe . . . . .                                                             | 5         |
| 2.3      | Uruchomienie kontenera . . . . .                                                          | 5         |
| 2.3.1    | Terminal systemu głównego . . . . .                                                       | 5         |
| 2.3.2    | Uruchomienie środowiska . . . . .                                                         | 6         |
| 2.4      | Połączenie przez przeglądarkę . . . . .                                                   | 7         |
| 2.5      | Praca w kontenerze . . . . .                                                              | 8         |
| <b>3</b> | <b>Konfiguracja sieciowa środowiska laboratoryjnego</b>                                   | <b>9</b>  |
| 3.1      | ROS_DOMAIN_ID – separacja komunikacji . . . . .                                           | 9         |
| 3.2      | Konfiguracja sieciowa robotów i stanowisk . . . . .                                       | 9         |
| 3.3      | Konfiguracja pliku <i>.bashrc</i> w kontenerze . . . . .                                  | 11        |
| 3.3.1    | Edycja pliku <i>.bashrc</i> . . . . .                                                     | 12        |
| <b>4</b> | <b>Uruchomienie środowiska Jupyter Notebook w Dockerze</b>                                | <b>13</b> |
| 4.1      | Uruchamianie Jupyter Notebook . . . . .                                                   | 13        |
| 4.2      | Problemy przy uruchamianiu . . . . .                                                      | 13        |
| 4.3      | Struktura katalogów w Jupyter Notebook . . . . .                                          | 13        |
| 4.3.1    | Uruchamianie ćwiczeń . . . . .                                                            | 14        |
| <b>5</b> | <b>Obsługa Jupyter Notebook</b>                                                           | <b>14</b> |
| 5.1      | Zapisywanie zmian . . . . .                                                               | 14        |
| 5.2      | Ikony i przyciski w pasku narzędzi . . . . .                                              | 15        |
| 5.3      | Typy komórek . . . . .                                                                    | 15        |
| 5.4      | Numeracja komórek . . . . .                                                               | 16        |
| 5.5      | <b>Komórki z <i>In [*]</i> – ważne ostrzeżenie</b> . . . . .                              | 16        |
| <b>6</b> | <b>Uruchomienie robota</b>                                                                | <b>17</b> |
| 6.1      | Przygotowanie robota . . . . .                                                            | 17        |
| 6.2      | Połączenie z robotem . . . . .                                                            | 18        |
| 6.3      | Konfiguracja środowiska na robocie . . . . .                                              | 18        |
| <b>7</b> | <b>Sterowanie robotem</b>                                                                 | <b>19</b> |

|          |                                                    |           |
|----------|----------------------------------------------------|-----------|
| 7.1      | Interfejs graficzny w Jupyter Notebook . . . . .   | 19        |
| 7.2      | Sterowanie z terminala . . . . .                   | 20        |
| 7.3      | Interfejs graficzny <i>rqt_publisher</i> . . . . . | 20        |
| <b>8</b> | <b>Weryfikacja działania kinematyki</b>            | <b>21</b> |
| 8.1      | Weryfikacja kinematyki odwrotnej . . . . .         | 21        |
| 8.2      | Weryfikacja kinematyki prostej . . . . .           | 22        |

# 1 Wprowadzenie

## 1.1 Przydatne skróty klawiaturowe

Podczas pracy w laboratorium często korzystamy z terminala, Jupyter Notebooka i edytorów tekstu. Należy rozróżnić skróty działające w systemie głównym od skrótów używanych wewnątrz środowiska Dockera. Skróty systemu Linux nie otwierają terminala kontenera.

Tab. 1.1: Przydatne skróty klawiaturowe

| Skrót        | Gdzie działa                        | Znaczenie / zastosowanie                            | Uwagi praktyczne                                       |
|--------------|-------------------------------------|-----------------------------------------------------|--------------------------------------------------------|
| Ctrl+Alt+T   | System główny Linux                 | Otwiera terminal systemu głównego                   | Nie otwiera terminala w Dockerze.                      |
| Ctrl+Shift+C | Terminal                            | Kopiuje zaznaczony tekst w terminalu                | Zwykle Ctrl+C w terminalu przerywa działanie programu. |
| Ctrl+Shift+V | Terminal                            | Wkleja tekst do terminala                           | Do wklejania poleceń w terminalu nie używamy Ctrl+V.   |
| Ctrl+C       | Instrukcja, Jupyter, pliki tekstowe | Kopiowanie zaznaczonego tekstu                      | Standardowy skrót w aplikacjach graficznych.           |
| Ctrl+V       | Jupyter, edytory tekstu             | Wklejanie skopiowanego tekstu                       | Działa w edytorach i przeglądarce.                     |
| Ctrl+/       | Jupyter Notebook                    | Komentowanie lub odkomentowywanie zaznaczonego kodu | Przydatne podczas testowania fragmentów kodu.          |
| Ctrl+Enter   | Jupyter Notebook                    | Uruchomienie zaznaczonej komórki                    | Kursor pozostaje w tej samej komórce.                  |
| Shift+Enter  | Jupyter Notebook                    | Uruchomienie komórki i przejście do kolejnej        | Najczęściej używany skrót podczas pracy z notatnikiem. |

## 1.2 Współdzielenie zasobów pomiędzy komputerem natywnym a środowiskiem w kontenerze

Podczas zajęć laboratoryjnych korzystamy ze środowiska uruchamianego w kontenerze Dockera. Wspólny katalog umożliwia przenoszenie plików pomiędzy systemem głównym komputera a środowiskiem otwartym w przeglądarce.

### 1.2.1 Wspólny katalog

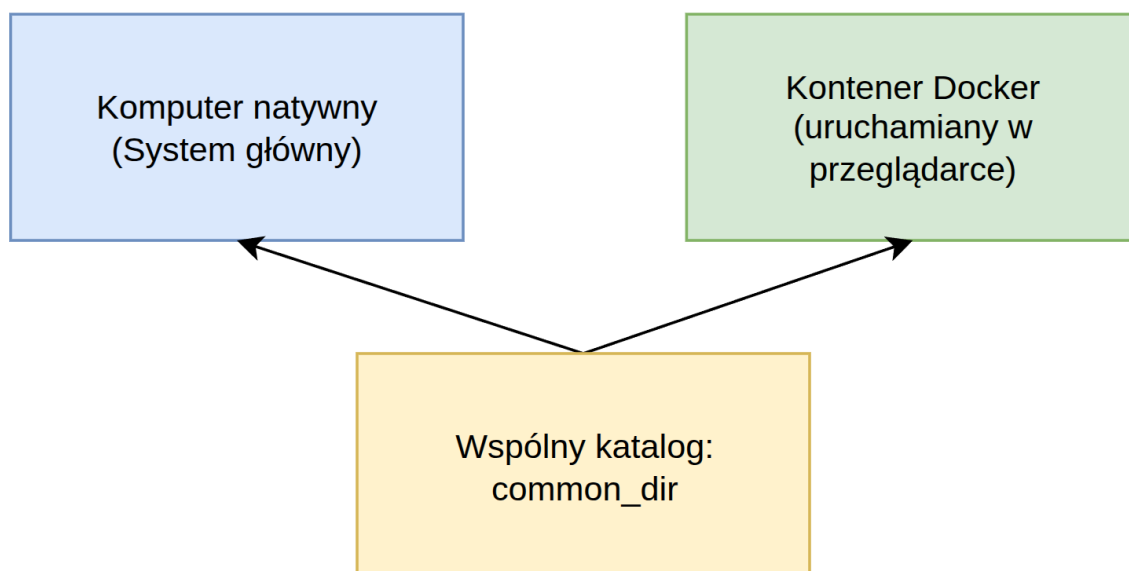
Na Pulpicie komputera w systemie głównym znajduje się katalog:

---

```
1 materials_do_zajec/roboty_mobilne/common_dir
```

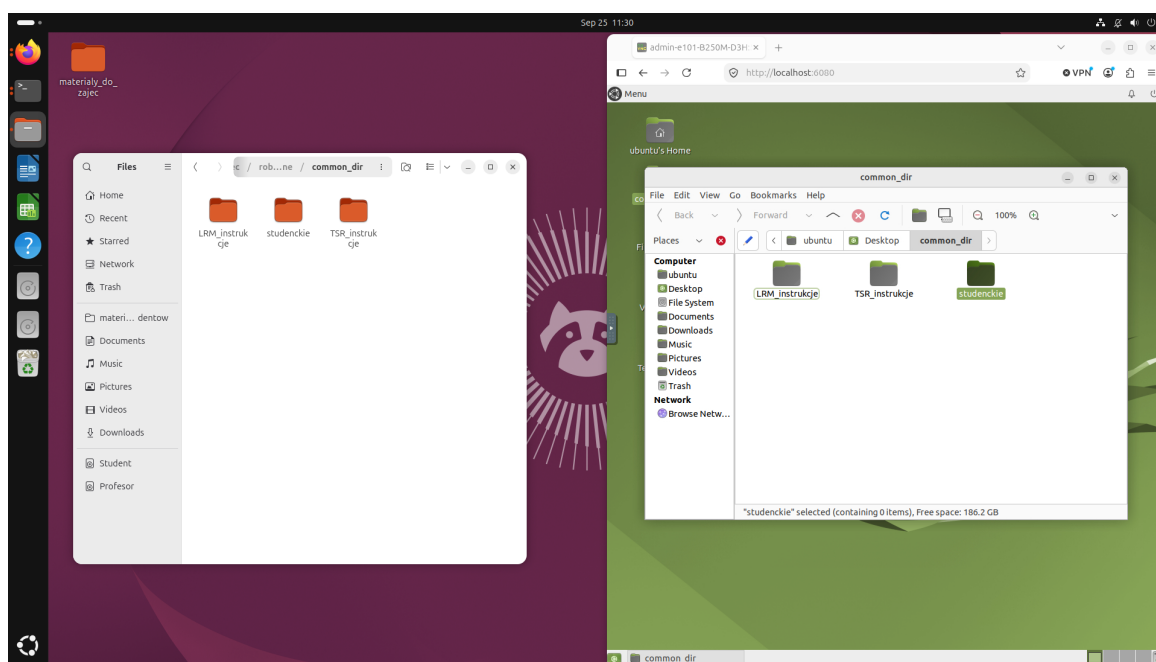
---

Kod źródłowy 1: Lokalizacja współdzielonego katalogu



Rys. 1.1: Schemat współdzielenia katalogu *common\_dir* pomiędzy hostem a Dockerem

Katalog *common\_dir* jest widoczny w obu środowiskach. Plik zapisany w nim na hoście jest od razu dostępny w Dockerze, a plik zapisany w Dockerze pojawia się na hoście. Schemat na Rys. 1.1 przedstawia wspólny katalog jako punkt wymiany plików pomiędzy systemem głównym a kontenerem. Układ katalogów i ich zawartość są takie same po obu stronach, co pokazano na Rys. 1.2.



Rys. 1.2: Ten sam katalog *common\_dir* widoczny na hoście i w Dockerze

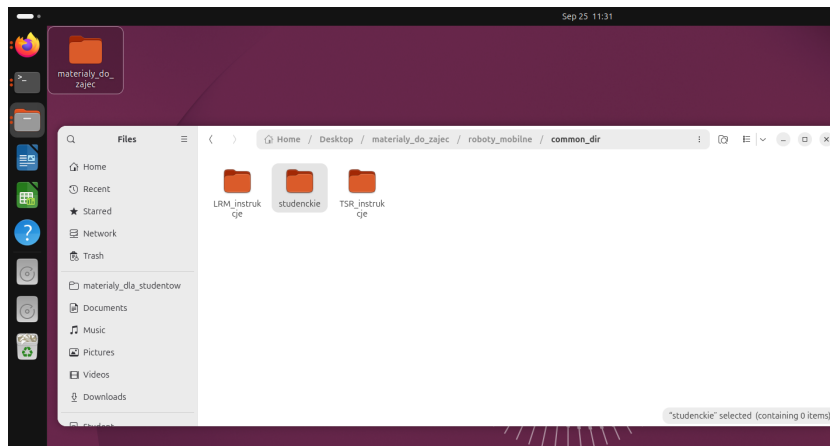
**Pliki, które mają być dostępne zarówno na hoście, jak i w Dockerze, należy zapisywać w katalogu *common\_dir*.**

## 1.2.2 Organizacja pracy studentów

Przed rozpoczęciem zajęć każdy student powinien:

1. Otworzyć katalog *common\_dir* na hoście i utworzyć w nim własny katalog wewnątrz folderu *studenckie*. Ten katalog jest miejscem przeznaczonym na własne pliki studenta.

2. Skopiować katalog **LRM\_instrukcje** z **common\_dir** do własnego katalogu w **studenckie**. Materiały są już dostępne na hoście; nie należy pobierać ich z platformy ani próbować kopiować ich z poziomu Dockera.



Rys. 1.3: Katalog **studenckie** na hoście przeznaczony na pliki studentów

**Nie należy modyfikować katalogu przykładowego z materiałami. W razie wykrycia zmiany zostanie on wyczyszczony i przywrócony do ustawień domyślnych, a wszystkie zapisane w nim zmiany zostaną utracone. Ostrzeżenie nie dotyczy własnego katalogu studenta w folderze **studenckie**.**

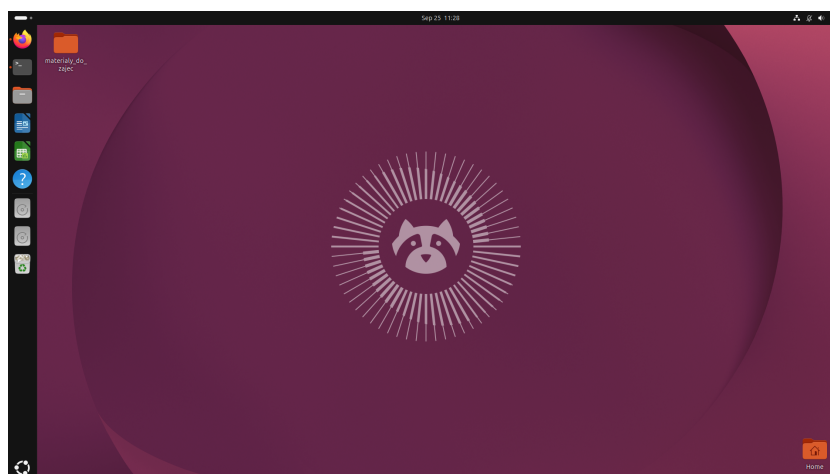
## 2 Uruchomienie kontenera

Poniżej opisano sposób uruchomienia środowiska laboratoryjnego działającego w kontenerze Dockera.

### 2.1 Logowanie do systemu

Po uruchomieniu komputera właściwy system uruchamia się automatycznie. Następnie należy zalogować się na konto:

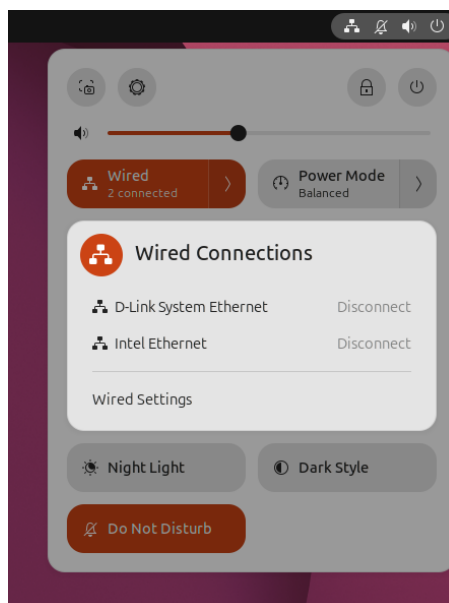
- nazwa użytkownika: **student**,
- hasło: **student**.



Rys. 2.1: Pulpit systemu głównego po zalogowaniu na konto **student**

## 2.2 Połączenia sieciowe

Po zalogowaniu oba połączenia są uruchamiane automatycznie. Nie należy ich ręcznie rozłączać ani zmieniać ich konfiguracji. Prawidłowy stan, z widocznymi aktywnymi połączeniami, przedstawia Rys. 2.2.



Rys. 2.2: Aktywne połączenia po zalogowaniu

## 2.3 Uruchomienie kontenera

### 2.3.1 Terminal systemu głównego

Terminal w systemie Ubuntu umożliwia komunikację z systemem operacyjnym za pomocą **wiersza poleceń**. Otwórz go ikoną przedstawioną na Rys. 2.3. Nie trzeba przechodzić do żadnego katalogu.



Rys. 2.3: Ikona uruchamiająca terminal systemu głównego

Terminal pozwala między innymi:

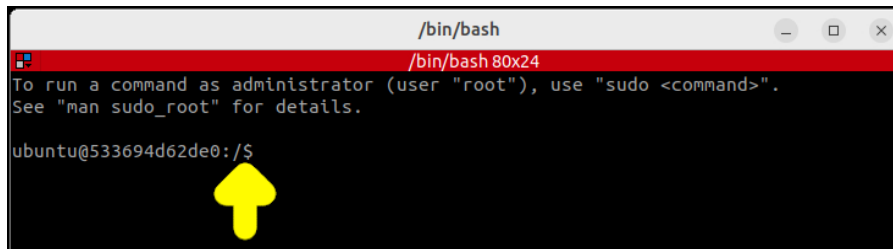
- uruchamiać programy,
- wykonywać polecenia administracyjne,

- pisać i uruchamiać skrypty,
- konfigurować środowisko pracy.

Podstawowym elementem terminala jest znak zachęty (*ang. prompt*). Jest to symbol lub tekst, który informuje, że terminal jest gotowy do przyjęcia polecenia. Terminal może znajdować się w dwóch stanach:

### 1. Tryb wprowadzania poleceń

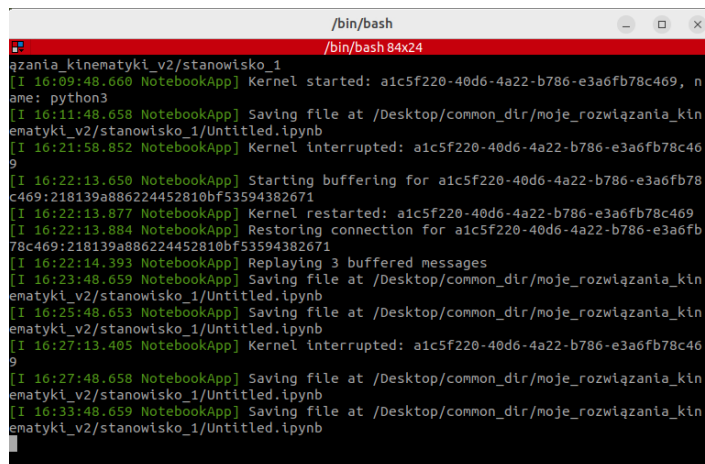
- widoczny jest znak zachęty i migający kursor, jak na Rys. 2.4,
- można wpisywać polecenia i zatwierdzać je klawiszem **Enter**,
- wynik każdego polecenia jest wyświetlany w tym samym oknie.



Rys. 2.4: Terminal gotowy do wprowadzania poleceń

### 2. Tryb pracy programu

- znak zachęty nie jest widoczny,
- terminal wyświetla wynik działania uruchomionego programu, na przykład logi systemowe, aplikację pełnoekranową albo narzędzie interaktywne,
- klawiatura służy do sterowania działającym procesem, a nie do wpisywania kolejnych poleceń.



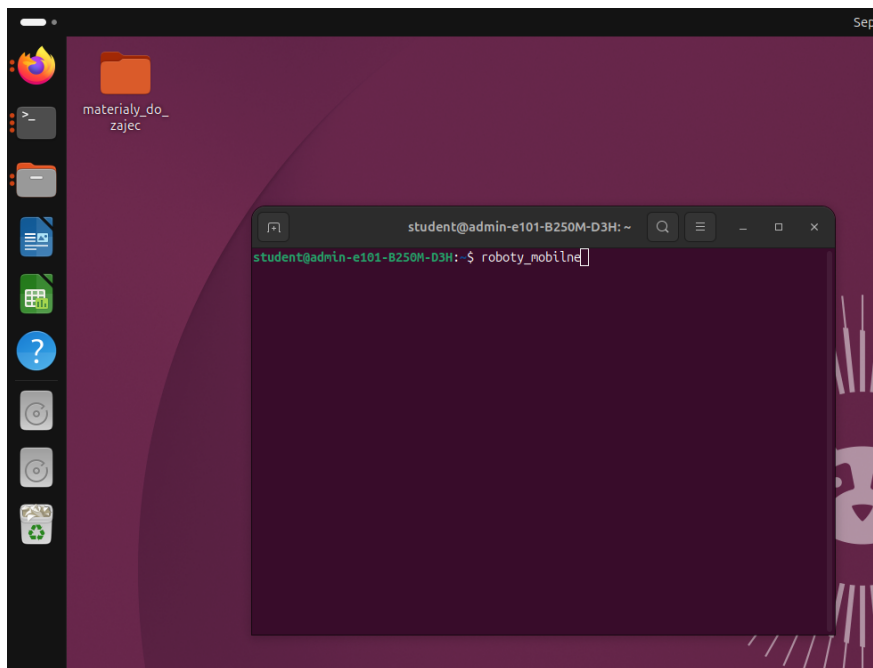
Rys. 2.5: Terminal podczas pracy programu

## 2.3.2 Uruchomienie środowiska

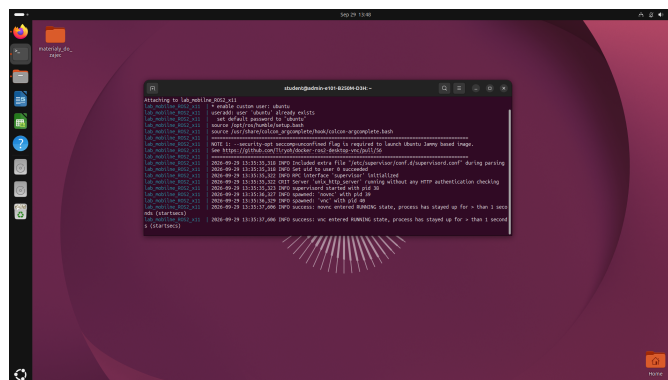
W terminalu wpisz alias:

1 roboty\_mobilne

Kod źródłowy 2: Alias uruchamiający kontener

Rys. 2.6: Wpisanie aliasu **roboty\_mobilne** w terminalu

Po zatwierdzeniu polecenia klawiszem **Enter** terminal przejdzie do trybu pracy programu. Rozpocznie się uruchamianie środowiska, a w terminalu pojawią się komunikaty startowe. Terminal należy pozostawić uruchomiony przez cały czas pracy z kontenerem.



Rys. 2.7: Terminal po uruchomieniu kontenera

**Gdy terminal wyświetla komunikaty działania programu, nie należy wpisywać w nim kolejnych poleceń. Aby powrócić do wpisywania poleceń, otwórz nowy terminal albo zakończ działanie bieżącego programu, na przykład skrótem **Ctrl+C**.**

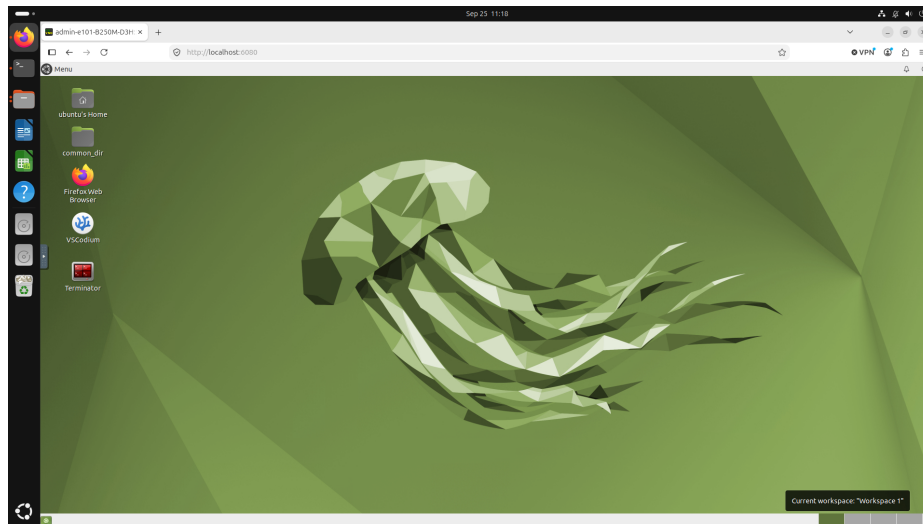
## 2.4 Połączenie przez przeglądarkę

1. Uruchom przeglądarkę **Firefox**.
2. W pasku adresu wpisz:

```
1 localhost:6080
```

Kod źródłowy 3: Adres środowiska Docker w przeglądarce

3. Kliknij przycisk **Połącz**. Po nawiązaniu połączenia widok przeglądarki powinien odpowiadać Rys. 2.8.

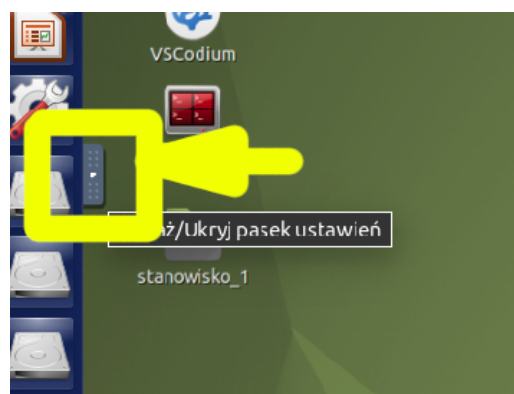


Rys. 2.8: Środowisko Dockera po połączeniu w przeglądarce Firefox

**Kontener został poprawnie uruchomiony – od tej chwili pracujemy w środowisku kontenera.**

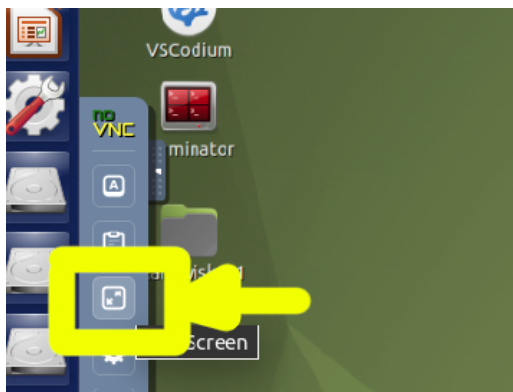
### 2.5 Praca w kontenerze

- Skrót **Ctrl+Alt+T** nie otwiera terminala Dockera; dotyczy on systemu głównego.
- Aby otworzyć terminal w kontenerze, kliknij prawym przyciskiem myszy na pulpicie i wybierz **Open in Terminal** albo uruchom program **Terminator** z pulpitu.
- Można przełączyć widok na pełny ekran, aby ukryć elementy systemu gospodarza i skupić się na środowisku kontenera.
- Z lewej strony ekranu kontenera widoczny jest pasek boczny albo mała strzałka, którą należy rozwinąć.



Rys. 2.9: Strzałka rozwijająca opcje środowiska kontenera

- Po rozwinięciu paska wybierz trzecią opcję od góry, aby uruchomić środowisko pracy w trybie pełnoekranowym.



Rys. 2.10: Opcja uruchamiająca tryb pełnoekranowy środowiska kontenera

## 3 Konfiguracja sieciowa środowiska laboratoryjnego

Podczas pracy w laboratorium z kilkoma robotami jednocześnie niezwykle istotne jest prawidłowe skonfigurowanie parametrów komunikacyjnych ROS 2. Dzięki temu dane przesyłane pomiędzy robotem a komputerem PC nie będą się mieszały z komunikacją innych stanowisk.

### 3.1 ROS\_DOMAIN\_ID – separacja komunikacji

W ROS 2 komunikacja między węzłami odbywa się dzięki mechanizmowi **DDS (Data Distribution Service)**. Każdy zestaw urządzeń (robot + komputer PC) może działać w osobnej **domenie komunikacyjnej**, oznaczonej parametrem **ROS\_DOMAIN\_ID**.

- Jeśli dwa systemy mają różne **ROS\_DOMAIN\_ID**, ich komunikacja jest rozdzielona – nawet jeśli używają takich samych nazw tematów (topics).
- Dzięki temu w laboratorium możliwa jest równoczesna praca wielu robotów bez ryzyka kolizji komunikacji.
- Przykład: przy 5 robotach w tym samym laboratorium każda grupa pracuje w innej domenie, więc np. temat `/cmd_vel` jednego robota nie zakłóca działania pozostałych.
- **UWAGA! Jeśli robot zaczął się poruszać, a nie powinien, istnieje ryzyko, że inna grupa nieprawidłowo skonfigurowała ROS\_DOMAIN\_ID. W takiej sytuacji można zapytać inne grupy, czy nie próbują uruchomić programu i czy ich robot działa poprawnie. Jeśli nie, powinny one ponownie zweryfikować, czy stanowisko zostało uruchomione prawidłowo.**

### 3.2 Konfiguracja sieciowa robotów i stanowisk

W laboratorium przygotowano pięć stanowisk, z których każde korzysta z innej platformy robota mobilnego:

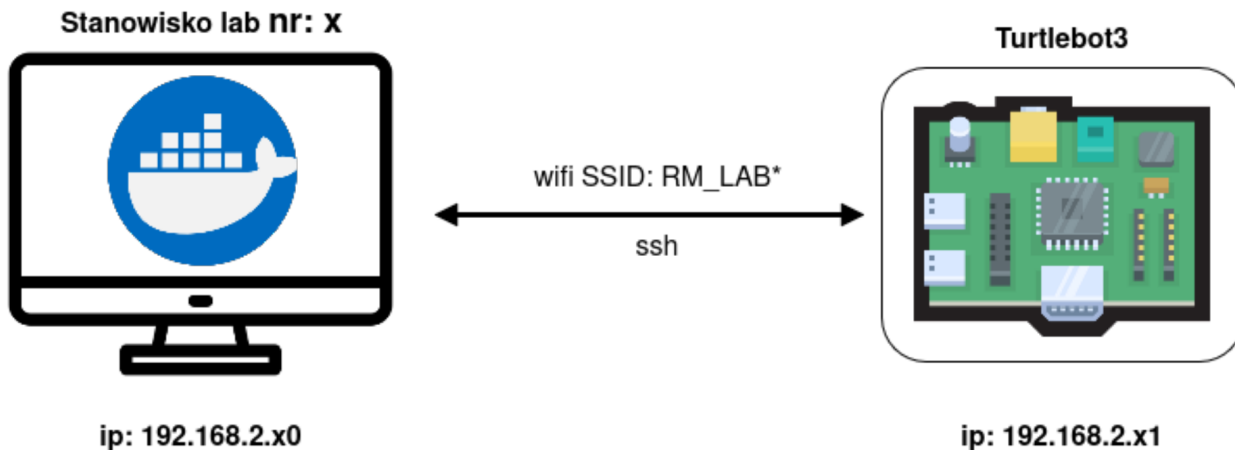
- Stanowisko 1 – robot z napędem różnicowym *differential drive*,
- Stanowisko 2 – robot z napędem typu *skid-steer*,

- Stanowisko 3 – robot z napędem *car-like* (skręcane koła przednie),
- Stanowisko 4 – robot z napędem *omni* (*swerve drive*),
- Stanowisko 5 – robot z napędem *omni* (*mecanum*).

Dla każdego stanowiska przypisane są konkretne adresy **IP** oraz numery **ROS\_DOMAIN\_ID**. Grupa powinna znać:

- **IP** oraz **ROS\_DOMAIN\_ID** robota (Turtlebot3),
- **ROS\_DOMAIN\_ID** przypisany do stanowiska PC.

Schemat na Rys. 3.1 przedstawia połączenie między stanowiskiem a robotem. Wartości adresów IP i **ROS\_DOMAIN\_ID** należy dobrać zgodnie z tabelą konfiguracyjną na Rys. 3.2.



Rys. 3.1: Schemat komunikacji z robotem.

#### Jak odczytać schemat:

- **x** oznacza numer stanowiska laboratoryjnego,
- komputer stanowiska ma adres **192.168.2.x0**, a robot adres **192.168.2.x1**,
- komputer i robot na tym samym stanowisku muszą mieć zgodną wartość **ROS\_DOMAIN\_ID**,
- połączenie sieciowe wykorzystuje Wi-Fi **RM\_LAB**, a dostęp do robota odbywa się przez **SSH**.

| Urządzenie                                                                                        | IP           | ROS_DOMAIN_ID |
|---------------------------------------------------------------------------------------------------|--------------|---------------|
| Główny router Teltonika RUTM10                                                                    | 192.168.2.1  |               |
| PC stanowisko 1                                                                                   | 192.168.2.10 | 10            |
| PC stanowisko 2                                                                                   | 192.168.2.20 | 20            |
| PC stanowisko 3                                                                                   | 192.168.2.30 | 30            |
| PC stanowisko 4                                                                                   | 192.168.2.40 | 40            |
| PC stanowisko 5  | 192.168.2.50 | 50            |
| TB3_1 RPi4<br>(2 wheel diff)                                                                      | 192.168.2.11 | 10            |
| TB3_2 RPi4<br>(skid-steer 4 wheels)                                                               | 192.168.2.21 | 20            |
| TB3_3 RPi4<br>(car like 4 wheels)                                                                 | 192.168.2.31 | 30            |
| TB3_4 RPi4<br>(swerve drive 4 wheels)                                                             | 192.168.2.41 | 40            |
| TB3_5 RPi4<br>(mecanum 4 wheels)                                                                  | 192.168.2.51 | 50            |
| TB4_1 RPi4<br>(Livox 360)                                                                         | 192.168.2.12 | 10            |
| TB4_2 RPi4<br>(OAK camera)                                                                        | 192.168.2.22 | 20            |
| TB4_3 RPi4                                                                                        | 192.168.2.32 | 30            |
| TB4_4 RPi4                                                                                        | 192.168.2.42 | 40            |
| TB4_5 RPi4                                                                                        | 192.168.2.52 | 50            |

Rys. 3.2: Konfiguracja sieci w laboratorium.

### 3.3 Konfiguracja pliku *.bashrc* w kontenerze

Plik *./bashrc* to skrypt wykonywany przy starcie każdej nowej sesji terminala w Bashu – służy do ustawiania zmiennych środowiskowych i aliasów. Dzięki wpisaniu tam np. `export ROS_DOMAIN_ID=30`, każdorazowe otwarcie terminala od razu ustawia właściwą domenę komunikacyjną ROS 2, co eliminuje potrzebę ręcznego konfigurowania przed uruchamianiem węzłów (nodów). Należy pamiętać, że jeśli w danym terminalu wykonujemy edycję tego pliku to, aby zmiany były widoczne należy uruchomić nowy terminal lub wpisać w aktualnym terminalu:

```
1 source ~/.bashrc
```

Kod źródłowy 4: Ponowne wczytanie pliku konfiguracyjnego *./bashrc*

### 3.3.1 Edycja pliku `.bashrc`

1. Otworzyć plik w edytorze **nano**:

```
1 nano ~/.bashrc
```

Kod źródłowy 5: Otwarcie pliku `./bashrc` w edytorze nano

2. W pliku odnaleźć linię zawierającą **ROS\_DOMAIN\_ID**.

- Jeśli jest zakomentowana (`#export ROS_DOMAIN_ID=...`), należy usunąć znak `#`.
- Jeśli nie istnieje, należy dopisać ją ręcznie w dowolnym miejscu pliku. Najlepiej na końcu.

```

/bin/bash
GNU nano 6.2 /home/ubuntu/.bashrc *
export GAZEBO_MODEL_PATH=$GAZEBO_MODEL_PATH:/home/ubuntu/turtlebot3_ws/src/turtlebot3_simulations/turtlebot3_ga
# TB3
source ~/turtlebot3_ws/install/setup.bash
export TURTLEBOT3_MODEL=burger # burger waffle waffle_pi
export LDS_MODEL=LDS-01
export ROS_DOMAIN_ID=30 #TURTLEBOT3; Ustawienie domeny dla robota, aby nie zakłócał komunikacji z innym robotem
#tCreate
source ~/create3_ws/install/setup.bash
export IGNITION_VERSION=fortress
source /usr/share/colcon_argcomplete/hook/colcon-argcomplete.bash
source ~/ros_ws/install/setup.bash
# Inne ustawienia
source ~/tsr_workspaces/example_tsr_ws/install/setup.bash
  
```

Rys. 3.3: Otwarty plik `./bashrc` z zaznaczoną przykładową linią z ustawionym **ROS\_DOMAIN\_ID**.

3. Zwrócić uwagę na kolejność wpisów – jeśli w pliku zmienna **ROS\_DOMAIN\_ID** występuje kilka razy, ostatnia wartość jest nadrzędna.
4. Przykład: jeśli najpierw ustawiono **ROS\_DOMAIN\_ID=20**, a później **ROS\_DOMAIN\_ID=50**, to faktycznie aktywna będzie wartość **50**.
5. Zapisać zmiany i wyjść z edytora:
  - **Ctrl+X**,
  - następnie wpisać **y** (yes) i zatwierdzić **Enter**.

6. Aby zmiany były widoczne w aktualnym terminalu, należy wpisać:

```
1 source ~/.bashrc
```

Kod źródłowy 6: Ponowne wczytanie pliku konfiguracyjnego `./bashrc`

lub otworzyć nowy terminal.

Po tej konfiguracji system będzie zawsze uruchamiał nowy terminal z poprawnym **ROS\_DOMAIN\_ID**, co gwarantuje poprawną komunikację z robotem przypisanym do stanowiska.

# 4 Uruchomienie środowiska Jupyter Notebook w Dockerze

**Jupyter Notebook** to interaktywne środowisko, które pozwala w jednym pliku pisać kod, wykonywać go krok po kroku oraz dokumentować wyniki. W ramach zajęć laboratoryjnych będziemy wykorzystywali język programowania **Python 3**.

## 4.1 Uruchamianie Jupyter Notebook

1. W terminalu w środowisku Dockera wpisz polecenie:

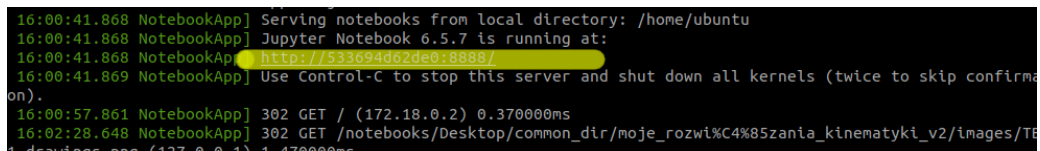
```
1 jupyter-notebook
```

Kod źródłowy 7: Polecenie służące do uruchomienia Jupyter Notebook.

2. Serwer Jupyter Notebook uruchomi się w kontenerze na porcie **8888**. Przeglądarka powinna otworzyć się automatycznie.

## 4.2 Problemy przy uruchamianiu

- Jeżeli po wpisaniu polecenia *jupyter-notebook* środowisko nie otworzy się automatycznie, w terminalu zostanie wyświetlony adres, pod którym Jupyter Notebook działa.
- Adres znajduje się w komunikacie wyświetlonym w terminalu, podobnym do pokazanego na Rys. 4.1. Zależy on od nazwy uruchomionego kontenera.



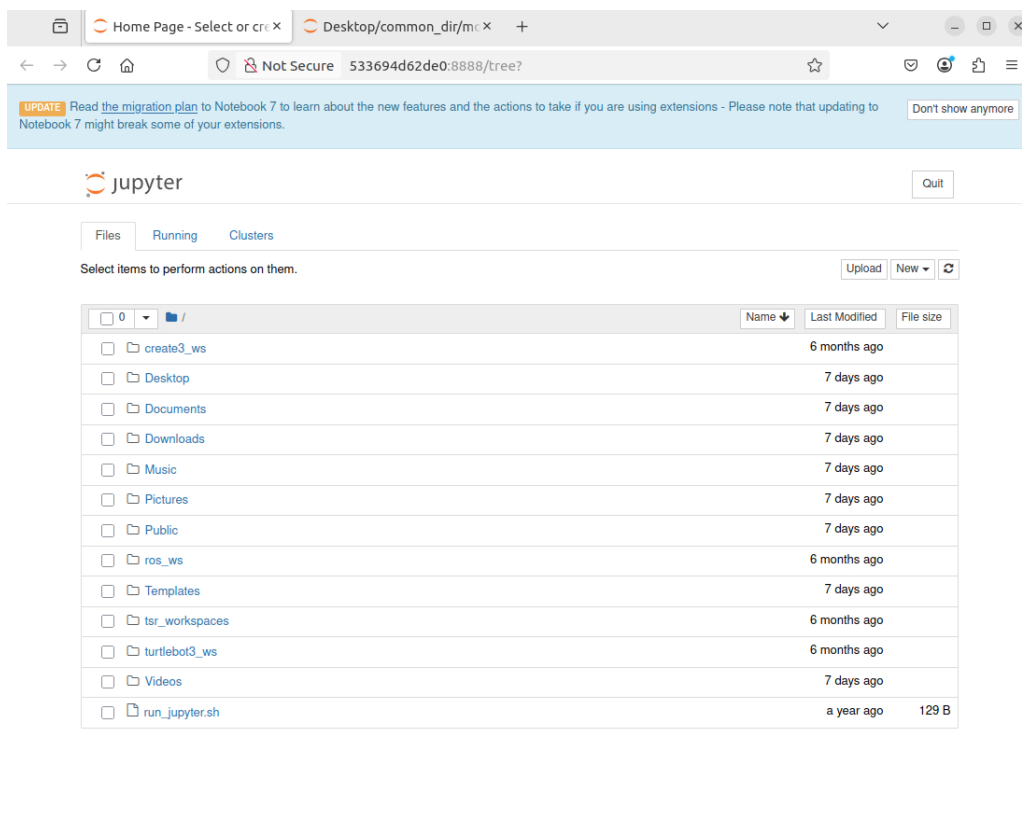
```
16:00:41.868 NotebookApp] Serving notebooks from local directory: /home/ubuntu
16:00:41.868 NotebookApp] Jupyter Notebook 6.5.7 is running at:
16:00:41.868 NotebookApp] http://533694d62de0:8888/
16:00:41.869 NotebookApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
16:00:57.861 NotebookApp] 302 GET / (172.18.0.2) 0.370000ms
16:02:28.648 NotebookApp] 302 GET /notebooks/Desktop/common_dir/ moje_rozwi%C4%85zania_kinematyki_v2/images/TB
  desktop.png (172.18.0.2) 1.470000ms
```

Rys. 4.1: Adres w terminalu dla Jupyter Notebook.

- Kliknij prawym przyciskiem myszy ten link i wybierz opcję **Otwórz w przeglądarce**.

## 4.3 Struktura katalogów w Jupyter Notebook

Po prawidłowym uruchomieniu notatnika widoczny będzie widok katalogów.



Rys. 4.2: Widok katalogów w Jupyter Notebook.

Należy przejść do:

```
1 Desktop/common_dir/studenckie
```

Kod źródłowy 8: Ścieżka do katalogu na prace studentów.

W tym miejscu powinien znajdować się wcześniej utworzony przez Państwa katalog.

### 4.3.1 Uruchamianie ćwiczeń

Ćwiczenia należy uruchamiać w kolejności – rozpoczynając od Ćwiczenia 1 przeznaczonego dla danego stanowiska.

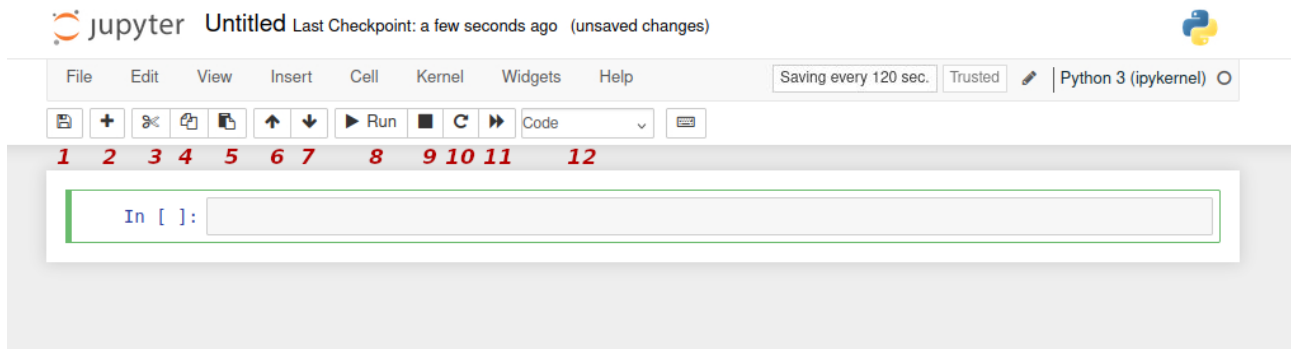
## 5 Obsługa Jupyter Notebook

W tym rozdziale przedstawiono najważniejsze elementy obsługi interfejsu **Jupyter Notebook** oraz typowe sytuacje, które mogą pojawić się podczas pracy.

### 5.1 Zapisywanie zmian

- Zmiany w notatniku zapisywane są **automatycznie**, ale można je też zapisać ręcznie.
- Jeśli w tytule pliku widnieje dopisek (**unsaved changes**), oznacza to, że nie wszystkie zmiany zostały zapisane.

## 5.2 Ikony i przyciski w pasku narzędzi



Rys. 5.1: Pasek narzędzi – Jupyter Notebook

1. **Zapisz** – przycisk do zapisu zmian
2. **+** – dodanie nowej komórki w notatniku.
3. **Nożyczki** – wycięcie zaznaczonej komórki (zostaje usunięta z widoku).
4. **Kopiuj** – skopiowanie zaznaczonej komórki.
5. **Wklej** – wklejenie wcześniej skopiowanej komórki poniżej aktualnie zaznaczonej.
6. **Strzałka w górę** – przesunięcie zaznaczonej komórki o jedną pozycję w górę.
7. **Strzałka w dół** – przesunięcie zaznaczonej komórki o jedną pozycję w dół.
8. **Run – uruchomienie komórki.**  
 Jeśli komórka zawiera kod (np. w Pythonie), zostaje on wykonany, a wynik pojawia się poniżej.  
 To najczęściej używana funkcja podczas zajęć.
9. **Interrupt the kernel** – przerywa aktualnie wykonywany kod, np. zatrzymuje nieskończoną pętlę.
10. **Restart the kernel** – uruchamia ponownie środowisko obliczeniowe (czyści pamięć i zmienne).
11. **Restart the kernel, then re-run the whole notebook** – restartuje środowisko i wykonuje wszystkie komórki od początku.  
 Przydatne przy ponownym uruchamianiu całego pliku. Należy jednak uważać na komórki z komendami dotyczącymi testów komunikacji z robotem – mogą wymagać zakomentowania, gdyż uruchamiają nieskończony proces oznaczony \*
12. Wybór typu komórki.

## 5.3 Typy komórek

- **Code** – do wpisywania i wykonywania kodu (np. w Pythonie). Wynik pojawia się pod spodem.
- **Markdown** – do opisu, dokumentacji i formatowanego tekstu (nagłówki, listy, wzory w LaTeX).
- **Raw** – zawiera surowy tekst, zapisywany bez przetwarzania.

W materiałach dostarczonych do zajęć większość komórek ma już ustawiony właściwy typ, jednak zawsze warto sprawdzić, czy kod uruchamiany jest w komórce typu **Code**.

## 5.4 Numeracja komórek

- Obok każdej komórki znajduje się pole **In [ ]**.
- Po uruchomieniu pierwszej komórki pojawia się numer **In [1]**.
- Kolejne uruchomienia są numerowane: **In [2]**, **In [3]**, itd.
- Numer oznacza, że kod został wykonany i można przejść do następnej komórki.



```
In [1]: a = 10.0

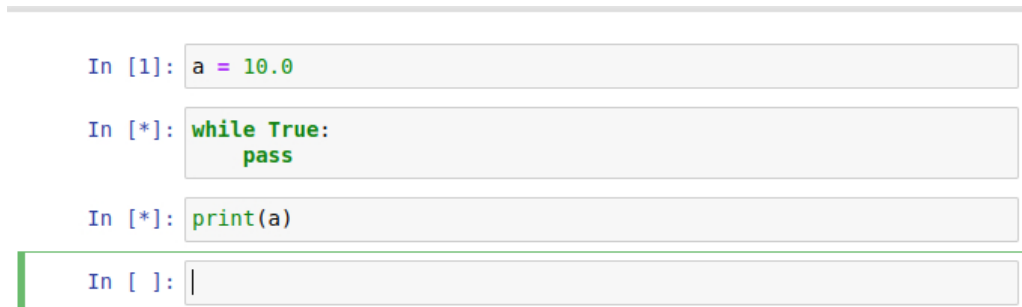
In [ ]: while True:
        pass
```

Rys. 5.2: Uruchomiona komórka **In [1]** w Jupyter Notebook.

## 5.5 Komórki z **In [\*]** – ważne ostrzeżenie

Uwaga na występowanie **In [\*]**, gdyż są one częstą przyczyną problemów studentów związanych z niedziałaniem uruchamianego kodu. Jeśli program został uruchomiony świadomie i powinien działać w pętli, nie jest to problem.

- Symbol **[\*]** oznacza, że kod w tej komórce ciągle się wykonuje.
- Dopóki nie zakończy się działanie programu, nie można uruchomić innych komórek.
- Najczęstsza przyczyna to nieskończona pętla.



```
In [1]: a = 10.0

In [*]: while True:
        pass

In [*]: print(a)

In [ ]: |
```

Rys. 5.3: Uruchomiona komórka **In [\*]** z nieskończoną pętlą w Jupyter Notebook.

W tym przypadku program nigdy się nie kończy – dlatego przy **In [ ]** pojawia się **[\*]**. Próba uruchomienia innej komórki (np. wyświetlenia zmiennej **a**) nie powiedzie się, dopóki pętla nie zostanie przerwana.

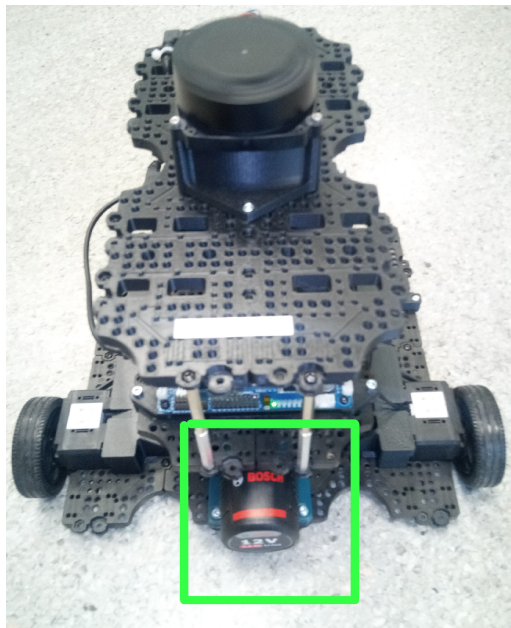
**Rozwiązanie: użyć przycisku Stop (Interrupt the kernel),  
a następnie ponownie uruchomić kod.**

## 6 Uruchomienie robota

Przed rozpoczęciem pracy z robotem należy pamiętać, że jego czas działania jest ograniczony przez baterię. Dlatego zanim włączysz **robota**, **przygotuj kod i środowisko**, które będą potrzebne podczas ćwiczenia.

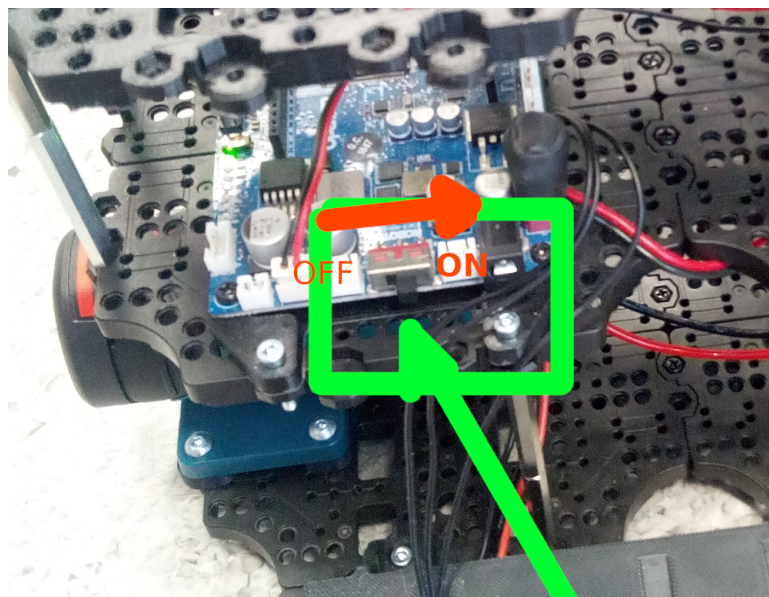
### 6.1 Przygotowanie robota

1. Umieścić naładowaną baterię w robocie.



Rys. 6.1: Umiejscowienie baterii w robocie.

2. Przełączyć przełącznik zasilania na niebieskiej płytce **OpenCR** w pozycję **ON** tzn. w prawo patrząc z przodu na przełącznik.



Rys. 6.2: Przełącznik na OpenCR

3. Robot rozpocznie proces uruchamiania – należy odczekać około **1 minuty**, aż pojawi się w sieci.

## 6.2 Połączenie z robotem

1. W środowisku Dockera otwórz terminal.
2. Połącz się z robotem, wpisując:

```
1 ssh pi@192.168.2.x1
```

Kod źródłowy 9: Przykładowe polecenia do połączenia z robotem

gdzie **x** oznacza numer stanowiska.

3. Hasło do logowania:

```
1 ubuntu
```

Kod źródłowy 10: Hasło do robota

4. Po poprawnym zalogowaniu terminal pokaże informacje o systemie.

```

pi@pi: ~
pi@pi: ~ 113x39
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-1063-raspi aarch64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Mon Mar 31 13:42:36 UTC 2025

System load:          0.08
Usage of /:            26.1% of 28.72GB
Memory usage:         5%
Swap usage:           0%
Temperature:          45.3 C
Processes:            138
Users logged in:      0
IPV4 address for wlan0: 192.168.2.11
IPV4 address for wlan0: 192.168.2.117
IPV6 address for wlan0: fd9f:f6f2:48e::727
IPV6 address for wlan0: fd9f:f6f2:48e:0:2ecf:67ff:fe5b:dfc1

Expanded Security Maintenance for Applications is not enabled.

11 updates can be applied immediately.
1 of these updates is a standard security update.
To see these additional updates run: apt list --upgradable

29 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection or proxy settings

Last login: Mon Mar 31 12:31:17 2025 from 192.168.2.113
pi@pi:~$

```

Rys. 6.3: Widok terminala po zalogowaniu się na robota.

Uwaga: łatwo odróżnić terminal Dockera od terminala robota:

- na robocie widać **pi@pi: \$**,
- w Dockerze widać **ubuntu@ciąg\_cyfr:\$**.

## 6.3 Konfiguracja środowiska na robocie

Tak samo jak w Dockerze, należy skonfigurować plik **.bashrc**:

1. Otwórz plik:

```
1 nano ~/.bashrc
```

Kod źródłowy 11: Otwarcie pliku **./bashrc** w edytorze nano.

2. W pliku odnaleźć linię zawierającą **ROS\_DOMAIN\_ID**.

- Jeśli jest zakomentowana (*#export ROS\_DOMAIN\_ID=...*), należy usunąć znak #.
- Jeśli nie istnieje, należy dopisać ją ręcznie w dowolnym miejscu pliku. Najlepiej na końcu.

---

```
1 export ROS_DOMAIN_ID=XX
```

---

Kod źródłowy 12: Linia w pliku */.bashrc* z konfiguracją **ROS\_DOMAIN\_ID**.

gdzie **XX** to numer przypisany do stanowiska – musi być taki sam jak w środowisku Dockera w */.bashrc*.

3. Zapisać zmiany i wyjść z edytora:

- **Ctrl+X**,
- następnie wpisać **y** (yes) i zatwierdzić **Enter**.

4. Aby zmiany były widoczne w aktualnym terminalu, należy wpisać:

---

```
1 source ~/.bashrc
```

---

Kod źródłowy 13: Ponowne wczytanie pliku konfiguracyjnego */.bashrc*

5. Aby uruchomić niezbędne węzły ROS 2 odpowiedzialne m.in. za czujniki i sterowanie napędami, wpisz w terminalu:

---

```
1 ros2 launch turtlebot3_bringup RM_lab.launch.py
```

---

Kod źródłowy 14: Launch służący do uruchomienia oprogramowania na robocie.

- Po tej komendzie robot zostaje uruchomiony.
- W tym terminalu **nie wpisujemy już żadnych innych poleceń** – będzie on wyświetlał logi działania robota.

**Robot jest gotowy do pracy.**

## 7 Sterowanie robotem

Sterowanie robotem może odbywać się na różne sposoby – zarówno za pomocą interfejsów graficznych, jak i z poziomu terminala. W ramach zajęć najczęściej wykorzystywany będzie **graficzny interfejs przygotowany w Jupyter Notebook**, ponieważ jest on najbardziej wygodny i pozwala w prosty sposób zadawać prędkości.

Szczegóły dotyczące wysyłanych wiadomości znajdują się w oddzielnej instrukcji 3 opisującej architekturę komunikacyjną pomiędzy robotem i innymi węzłami ROS uruchamianymi w trakcie zajęć.

Niezależnie od wybranej metody sterowanie zawsze odbywa się poprzez wysyłanie wiadomości typu *geometry\_msgs/msg/Twist* na temat */cmd\_vel*.

### 7.1 Interfejs graficzny w Jupyter Notebook

Należy uruchomić przygotowany interfejs sterowania znajdujący się w pliku *robot\_control\_interface.ipynb*.

Linear X:

Linear Y:

Angular Z:

Topic freq: Hz

Step sizes: Linear Step:  Angular Step:

Current Velocity:  
Linear: x=0.0, y=0.0  
Angular: z=0.0

```
[INFO] [1742318738.518295167] [velocity_publisher]: Velocity reset to zero.
[INFO] [1742318752.154663284] [velocity_publisher]: Velocity reset to zero.
[INFO] [1742318764.571995319] [velocity_publisher]: Velocity reset to zero.
```

Rys. 7.1: Sterowanie z Jupyter Notebook.

Interfejs dostępny w Jupyter Notebooku pozwala w intuicyjny sposób zadawać wartości prędkości liniowej i kątowej robota, bez konieczności wpisywania komend w terminalu.

## 7.2 Sterowanie z terminala

Robotem można sterować również ręcznie poprzez terminal. Przykład wysłania wiadomości na temat `/cmd_vel` z użyciem komendy `ros2 topic pub`:

```
1 ros2 topic pub /cmd_vel geometry_msgs/msg/Twist
2 linear:
3   x: 0.5
4   y: 0.0
5   z: 0.0
6 angular:
7   x: 0.0
8   y: 0.0
9   z: 0.2
```

Kod źródłowy 15: Wysłanie przykładowych prędkości sterujących do robota z poziomu terminala.

Ta komenda ustawia prędkość liniową 0.5 m/s oraz prędkość obrotową 0.2 rad/s.

```
ubuntu@lukasz-ThinkPad-T480:~$ ros2 topic pub /cmd_vel geometry_msgs/msg/Twist "{linear: {x: 0.5, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.2}}"
publisher: beginning loop
publishing #1: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.5, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.2))
publishing #2: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.5, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.2))
publishing #3: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.5, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.2))
publishing #4: geometry_msgs.msg.Twist(linear=geometry_msgs.msg.Vector3(x=0.5, y=0.0, z=0.0), angular=geometry_msgs.msg.Vector3(x=0.0, y=0.0, z=0.2))
^Cubuntu@lukasz-ThinkPad-T480:~$
```

Rys. 7.2: Sterowanie z terminala.

## 7.3 Interfejs graficzny `rqt_publisher`

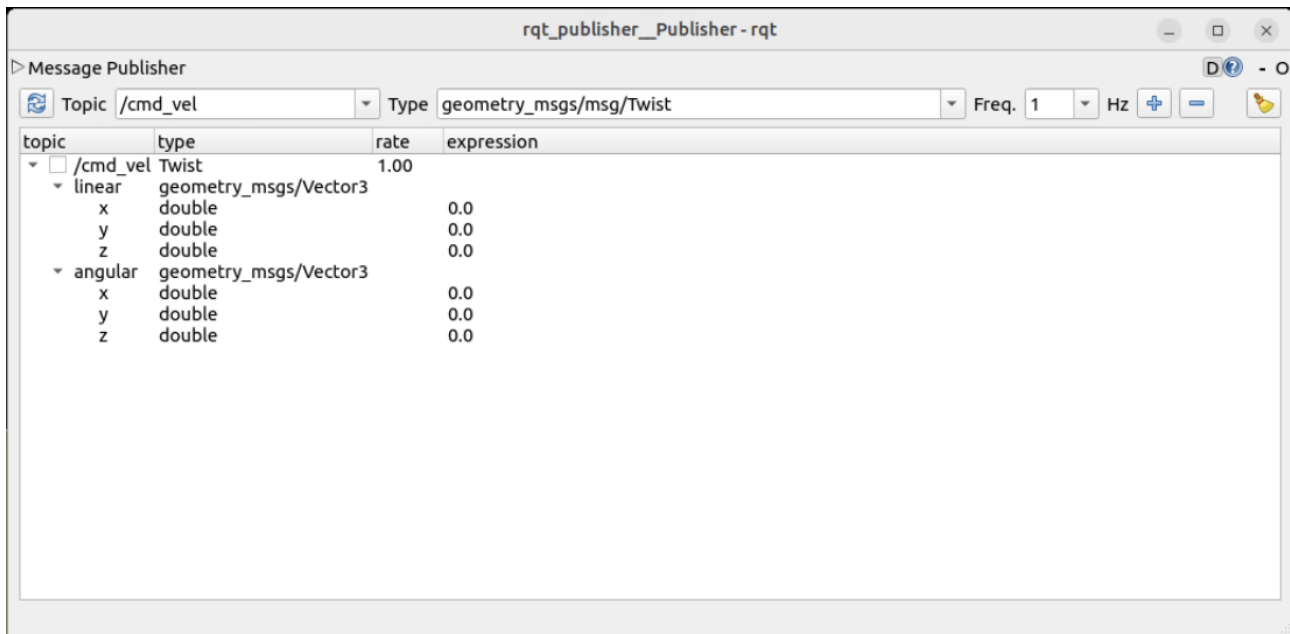
Innym sposobem jest wykorzystanie standardowego narzędzia ROS 2 – `rqt_publisher`.

Aby uruchomić program, wpisz w terminalu w kontenerze:

```
1 ros2 run rqt_publisher rqt_publisher
```

Kod źródłowy 16: Komenda w terminalu do uruchomienia `rqt_publisher`

Program otwiera graficzne okno, w którym można ręcznie wprowadzać wartości do wiadomości i wysyłać je do wybranego tematu. Z listy rozwijanej **Topic** należy wybrać **/cmd\_vel**



Rys. 7.3: Sterowanie z rqt\_publisher.

## 8 Weryfikacja działania kinematyki

Podczas pracy w środowisku ROS 2 można w prosty sposób sprawdzić, czy uruchomione oprogramowanie działa poprawnie i wymienia dane. Weryfikacja **nie sprawdza poprawności obliczeń**, ale pozwala upewnić się, że węzły zostały uruchomione i publikują odpowiednie komunikaty.

Wszystkie polecenia należy wpisywać w terminalu w **Dockerze**.

### 8.1 Weryfikacja kinematyki odwrotnej

1. Uruchom kod kinematyki odwrotnej (plik z wykonanym ćwiczeniem nr 2).
2. Wyślij prędkości sterujące robotem na temat **/cmd\_vel** np. poprzez interfejs graficzny Jupyter Notebook.
3. Otwórz nowy terminal i wpisz (krok opcjonalny):

```
1 ros2 topic echo /cmd_vel
```

Kod źródłowy 17: Nasłuchiwanie wiadomości ROS pojawiających się na temacie **/cmd\_vel**.

- To polecenie pozwala sprawdzić, czy interfejs użytkownika wysyła poprawnie prędkości sterujące robotem.
- Brak publikacji na tym temacie uniemożliwia obliczenie prędkości dla poszczególnych kół.

4. W kolejnym terminalu sprawdź, czy węzeł kinematyki odwrotnej publikuje przeliczone prędkości kół:

```
1 ros2 topic echo /cmd_joint_states
```

Kod źródłowy 18: Nasłuchiwanie wiadomości ROS pojawiających się na temacie **/cmd\_joint\_states**.

- Jeśli wszystko działa poprawnie, pojawią się wartości prędkości dla poszczególnych kół.

- Można zmienić prędkości sterujące w interfejsie i zaobserwować, czy zmieniają się wartości publikowane na temat `/cmd_joint_states`.

## 5. Jeśli wszystko działa poprawnie można uruchomić rozwiązanie na rzeczywistym robocie.

## 8.2 Weryfikacja kinematyki prostej

1. Uruchom kod kinematyki prostej (plik z wykonanym ćwiczeniem nr 3).
2. Sprawdź w terminalu, czy odbierane są dane z czujników kół (m.in. ich prędkości i pozycje):

```
1 ros2 topic echo /joint_states
```

Kod źródłowy 19: Nasłuchiwanie wiadomości ROS pojawiających się na temacie `/joint_states`.

- Brak danych oznacza, że odometria nie zostanie obliczona.

3. Sprawdź w terminalu, czy węzeł kinematyki prostej publikuje dane dotyczące odometrii:

```
1 ros2 topic echo /odometry
```

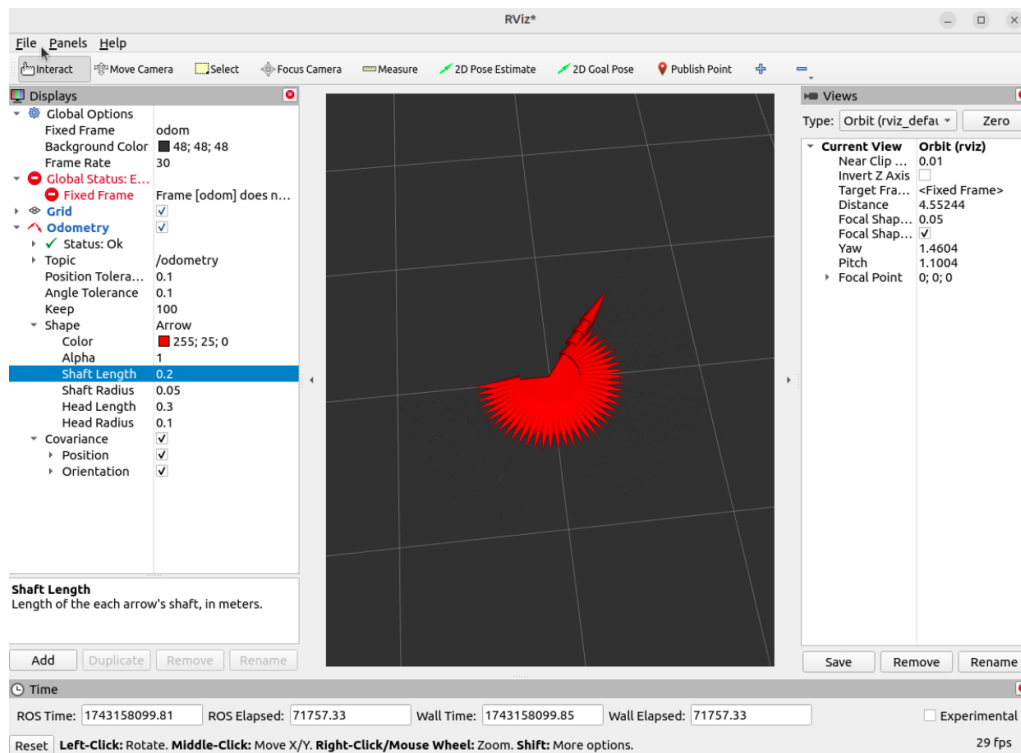
Kod źródłowy 20: Nasłuchiwanie wiadomości ROS pojawiających się na temacie `/odometry`.

- Jeżeli dane są widoczne, oznacza to, że węzeł działa poprawnie i oblicza pozycję robota na podstawie informacji z kół.

4. Zweryfikuj poprawność działania odometrii w sposób graficzny, uruchamiając program **RViz2**:

```
1 rviz2
```

Kod źródłowy 21: Uruchomienie programu **RViz2** do wizualizacji danych



Rys. 8.1: Wizualizacja danych za pomocą **RViz2**

- W polu **Global Options** w **Fixed Frame** ustaw ramkę `odom`, a jeśli nie istnieje, wpisz ją ręcznie.

- W oknie programu kliknij **Add** i wybierz temat */odometry*.
- Program wyświetli trajektorię ruchu robota, co pozwala ocenić poprawność wyliczania odometrii.
- **RViz2** stanowi wygodne uzupełnienie weryfikacji danych z terminala, gdyż umożliwia ich wizualizację.